Dasar Object Relational Maping

Studi Kasus Simple-LMS

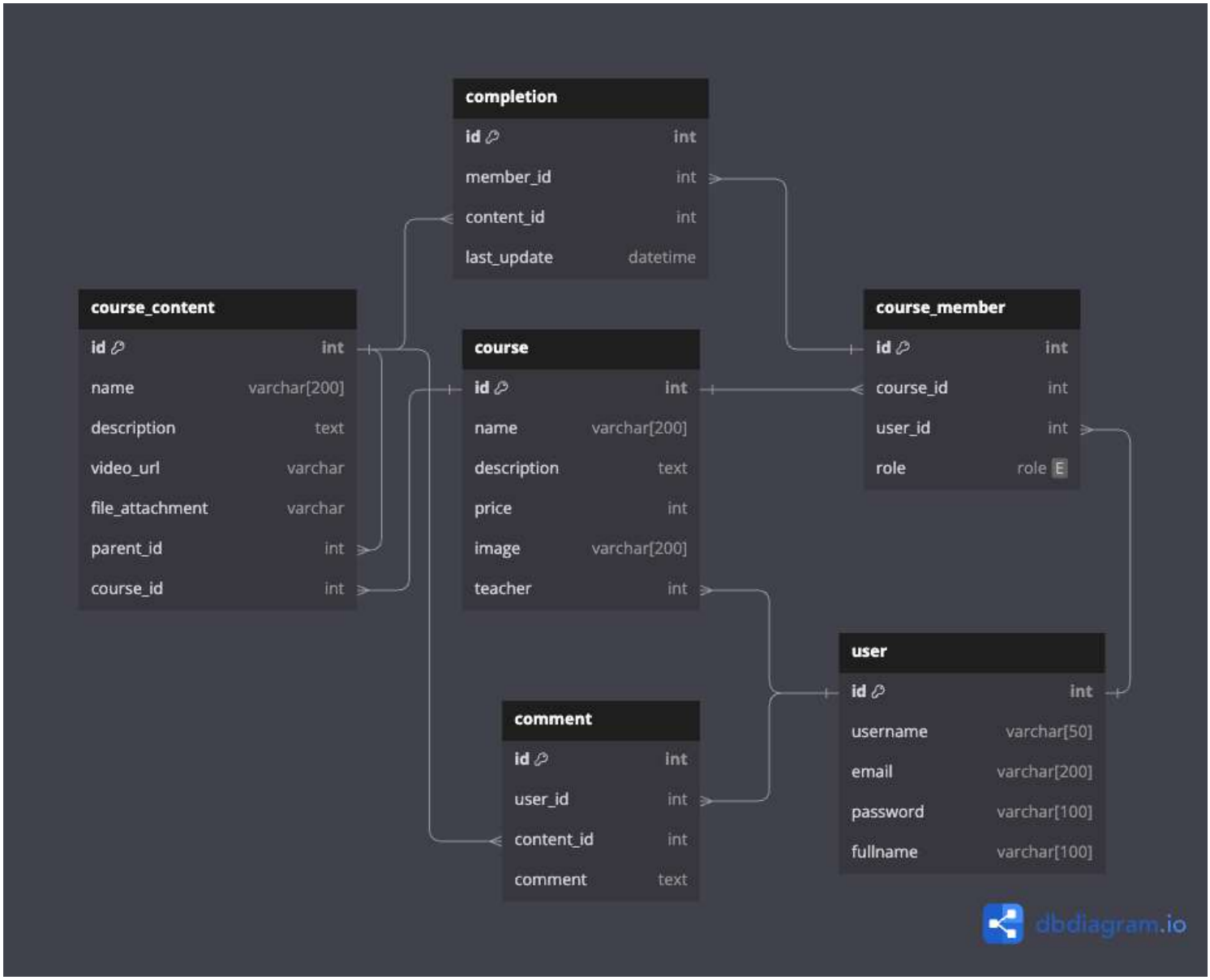
Pada bagian ini dan seterusnya (hingga penerapan capstone project), kita akan menggunakan contoh studi kasus pengembangan backend aplikasi simple Learning Management System (LMS). Aplikasi yang akan kita bangun ini mirip seperti backend untuk aplikasi Udemy dengan pemangkasan fitur tentu saja.

Secara garis besar, kebutuhan dari aplikasi tersebut adalah sebagai berikut:

- Aplikasi memungkinkan user untuk membuat mendaftar dan membuat course baru dengan mendefinisikan deskripsi serta harganya.
- Selanjutnya user tersebut dapat menambahkan juga content yang berupa teks, link video, serta attachment file.
- User lain dapat enroll pada course yang sudah dibuat agar dapat mengakses konten yang ada pada course tersebut.
- User yang sudah menjadi member pada suatu course, dapat berkomentar pada content tertentu untuk melakukan tanya jawab dan interaksi lain seputar konten tersebut.
- Serta, dapat menandai completion pada konten tertentu untuk tracking progress pada course yang diikuti.

Definisi Model untuk Schema

Berikut adalah diagram relasi tabel yang digunakan pada studi kasus Simple LMS:



Untuk membuat model dari skema tersebut kodingnya adalah seperti ini:

```python
from django.db import models
from django.contrib.auth.models import User

class Course(models.Model):
    name = models.CharField("nama matkul", max_length=100)
    description = models.TextField("deskripsi", default='-')
    price = models.IntegerField("harga", default=10000)
    image = models.ImageField("gambar", null=True, blank=True)
    teacher = models.ForeignKey(User, verbose_name="pengajar", on_delete=models.RESTRICT)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        verbose_name = "Mata Kuliah"
        verbose_name_plural = "Mata Kuliah"

```

```
    def __str__(self) -> str:
        return self.name+" : "+self.price


ROLE_OPTIONS = [('std', "Siswa"), ('ast', "Asisten")]

class CourseMember(models.Model):
    course_id = models.ForeignKey(Course, verbose_name="matkul", on_delete=models.RESTRICT)
    user_id = models.ForeignKey(User, verbose_name="siswa", on_delete=models.RESTRICT)
    roles = models.CharField("peran", max_length=3, choices=ROLE_OPTIONS, default='std')
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        verbose_name = "Subscriber Matkul"
        verbose_name_plural = "Subscriber Matkul"

    def __str__(self) -> str:
        return self.course_id+" : "+self.user_id

class CourseContent(models.Model):
    name = models.CharField("judul konten", max_length=200)
    description = models.TextField("deskripsi", default='-')
    video_url = models.CharField('URL Video', max_length=200, null=True, blank=True)
    file_attachment = models.FileField("File", null=True, blank=True)
    course_id = models.ForeignKey(Course, verbose_name="matkul", on_delete=models.RESTRICT)
    parent_id = models.ForeignKey("self", verbose_name="induk",
                                  on_delete=models.RESTRICT, null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        verbose_name = "Konten Matkul"
        verbose_name_plural = "Konten Matkul"

    def __str__(self) -> str:
        return '['+self.course_id+"] "+self.name


class Comment(models.Model):
    content_id = models.ForeignKey(CourseContent, verbose_name="konten", on_delete=models.CASCAD
    member_id = models.ForeignKey(CourseMember, verbose_name="pengguna", on_delete=models.CASCAD
    comment = models.TextField('komentar')
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        verbose_name = "Komentar"
        verbose_name_plural = "Komentar"

    def __str__(self) -> str:
```

```
68          return "Komen: "+self.content_id.name+"-"+self.user_id
```

Migrasi Schema

Di dalam Django ada 3 tahap migrasi skema yaitu:

1. Membuat koding modelnya
2. Generate skrip migrasi
3. Eksekusi skrip migrasi ke dalam basis data

Di tahap sebelumnya kita sudah membuat koding model yang merupakan representasi tabel basis data relasional, sehingga tahap selanjutnya yang perlu dilakukan adalah generate skrip migrasi. Untuk melakukan hal tersebut, kita bisa menjalankan perintah ini pada terminal:

```
docker exec simple_lms python manage.py makemigrations
```

Setelah menjalankan perintah tersebut, cek di direktori migrasi. File yang berisi skrip migrasi otomatis sudah dibuat pada direktori tersebu
Sebagai catatan, ketika terjadi perubahan skema (baik penambahan, perubahan, atau penghapusan), maka kita perlu mengeksekusi perin
tersebut lagi untuk di-generate-kan skrip perubahannya.

Langkah selanjutnya adalah eksekusi skrip migrasi dengan menjalankan perintah berikut pada terminal:

```
docker exec simple_lms python manage.pyt migrate
```

Query Dasar

Eksekusi perintah berikut pada views:

- Insert data user & Cek di DB viewer

```
User.objects.create_user(username="usertesting", email="usertest@email.com",
                         password="sanditesting")
```

- Insert data course & Cek di DB viewer

```
Course.objects.create(name=row['name'], description=row['description'],
                      price=row['price'], teacher=User.objects.get(pk=1))
```

- Select all data user
  - Tampilkan di HTML
  - Tampilkan melalui JSONResponse
- Select get
  - Tampilkan di HTML
- Select where user
- Update data user
- Delete data user
- Delete all course data
- Delete all user data except

```
def testing(request):
    user_test = User.objects.filter(username="usertesting")
    if not user_test.exists():
        user_test = User.objects.create_user(
                            username="usertesting",
                            email="usertest@email.com",
                            password="sanditesting")
    all_users = serializers.serialize('python', User.objects.all())

    admin = User.objects.get(pk=1)
    user_test.delete()

    after_delete = serializers.serialize('python', User.objects.all())

    response = {
            "admin_user": serializers.serialize('python', [admin])[0],
            "all_users" : all_users,
            "after_del" : after_delete,
        }
    return JsonResponse(response)
```

Input banyak data dengan import CSV

Kita akan melakukan importing data dummy ke dalam basis data degan menggunakan data CSV yang sudah disiapkan sebelumnya, untuk memudahkan simulasi dan mempelajari operasi lanjut dari Django ORM. Proses import CSV dapat dilakukan secara terpisah dari aplikasi django. Artinya kita membuat skrip python khusus di luar aplikasi utama untuk melakukan importing data CSV ini. Sehingga langkah yang perlu dilakukan adalah:

- Siapkan file CSV yang akan diimport dan masukkan pada direktori `code/csv_data`
- Buat 1 file baru bernama `importer.py` di direktori yang sama dengan file `manage.py`. Isi dari file tersebut adalah sebagai berikut:

```
import os
import sys
sys.path.append(os.path.abspath(os.path.join(__file__, *[os.pardir] * 3)))
os.environ['DJANGO_SETTINGS_MODULE'] = 'simplelms.settings'
import django
django.setup()

import csv
from django.contrib.auth.models import User
from core.models import Course, CourseMember

with open('./csv_data/user-data.csv') as csvfile:
        reader = csv.DictReader(csvfile)
        for num, row in enumerate(reader):
            if not User.objects.filter(username=row['username']).exists():
                User.objects.create_user(
                    id=num+2, username=row['username'],
                    password=row['password'],
```

```
                    email=row['email'])

with open('./csv_data/course-data.csv') as csvfile:
    reader = csv.DictReader(csvfile)
    for num,row in enumerate(reader):
        if not Course.objects.filter(pk=num+1).exists():
            Course.objects.create(
                    id=num+1, name=row['name'],
                    description=row['description'],
                    price=row['price'],
                    teacher=User.objects.get(pk=int(row['teacher'])))

with open('csv_data/member-data.csv') as csvfile:
    reader = csv.DictReader(csvfile)
    for num, row in enumerate(reader):
        if not CourseMember.objects.filter(pk=num+1).exists():
            CourseMember.objects.create(
                    course_id=Course.objects.get(pk=int(row['course_id'])),
                    user_id=User.objects.get(pk=int(row['user_id'])),
                    id=num+1, roles=row['roles'])
```

- Selanjutnya eksekusi perintah berikut pada terminal:

```
docker exec simpe_lms python import.py
```

Query Relasional

- Select course with teacher data

```
def allCourse(request):
    allCourse = Course.objects.all()
    result = []
    for course in allCourse:
        record = {'id': course.id, 'name': course.name,
                  'description': course.description,
                  'price': course.price,
                  'teacher': {
                      'id': course.teacher.id,
                      'username': course.teacher.username,
                      'email': course.teacher.email,
                      'fullname': f"{course.teacher.first_name} {course.teacher.last_name}"
                  }}
        result.append(record)
    return JsonResponse(result, safe=False)
```

- Select user data & his courses (as teacher)

```
def userCourses(request):
    user = User.objects.get(pk=3)
    courses = Course.objects.filter(teacher=user.id)
```

```
    course_data = []
    for course in courses:
        record = {'id': course.id, 'name': course.name,
                  'description': course.description, 'price': course.price}
        course_data.append(record)
    result = {'id': user.id, 'username': user.username, 'email': user.email,
              'fullname': f"{user.first_name} {user.last_name}",
              'courses': course_data}
    return JsonResponse(result, safe=False)
```

- Select course data with aggregate

```
def courseStat(request):
    courses = Course.objects.all()
    stats = courses.aggregate(max_price=Max('price'),
                              min_price=Min('price'),
                              avg_price=Avg('price'))
    result = {'course_count': len(courses), 'courses': stats}
    return JsonResponse(result, safe=False)
```

- Select filter course data with aggregate

```
def courseMemberStat(request):
    courses = Course.objects.filter(description__contains='python') \
                            .annotate(member_num=Count('coursemember'))
    course_data = []
    for course in courses:
        record = {'id': course.id, 'name': course.name, 'price': course.price,
                  'member_count': course.member_num}
        course_data.append(record)
    result = {'data_count': len(course_data), 'data':course_data}
    return JsonResponse(result)
```